

ICMFatturazioni — suite ICM Solutions

Consultazione verbali in ICMFatturazioni

Condivisione e messa a disposizione dei verbali di cantiere dal dominio ICMVerbalì

Versione	Data	Stato	Classificazione
1.0	09/07/2026	Rilasciato	Riservato — uso interno

A CURA DI

Vis Logica

Automazioni e software per PMI
vislogica.com

ANAGRAFICA DOCUMENTO

1 Informazioni sul documento

Titolo	Consultazione verbali in ICMFatturazioni
Oggetto	Condivisione e messa a disposizione dei verbali di cantiere dal dominio ICMVerbalì
Progetto / prodotto	ICMFatturazioni — suite ICM Solutions
A cura di	Vis Logica
Autore	Luca Schiavon
Versione	1.0
Data	09/07/2026
Stato	Rilasciato
Classificazione	Riservato — uso interno

Storico delle revisioni

Versione	Data	Autore	Descrizione delle modifiche
1.0	09/07/2026	Luca Schiavon	Prima stesura rilasciata: descrizione completa della maschera Consultazione verbali e delle logiche di condivisione dei PDF dal dominio ICMVerbalì.

Proprietà

Il presente documento è di proprietà di Vis Logica. Ne è vietata la riproduzione o distribuzione non autorizzata.

SOMMARIO

2 Indice

1 — Informazioni sul documento	2
2 — Indice	3
3 — Introduzione e obiettivi	4
4 — Contesto: database unificato e entità condivise	5
5 — Architettura della soluzione	6
6 — Il modello dati e la vista fatt.VerbaliConsultazione	8
7 — Data Access Layer (repository Dapper)	10
8 — Business logic: il Manager e il filtro di esistenza fisica	11
9 — Storage dei PDF sul filesystem condiviso	12
10 — Gli endpoint HTTP: apertura PDF e download ZIP	14
11 — L'interfaccia utente: filtro a cascata e griglia	16
12 — Sicurezza, permessi e messa in produzione	18
13 — Dependency Injection, configurazione e test	20
14 — Riepilogo e possibili estensioni	21

PREMESSA

3 Introduzione e obiettivi

La maschera **Consultazione verbali** (rotta `/consultazione-verbali`) permette agli utenti di **ICMFatturazioni** di cercare, aprire e scaricare i **verbali di cantiere** firmati, pur essendo questi ultimi un dominio di competenza dell'applicativo gemello **ICMVerbali**.

I due applicativi appartengono alla stessa suite di ICM Solutions e, dal 19/06/2026, condividono un **unico database** (`ICMVerbalIdb`). Questo apre la possibilità di far dialogare i due domini senza duplicare dati: ICMFatturazioni, che si occupa del ciclo di fatturazione, ha bisogno di consultare i verbali prodotti sul cantiere (per allegarli, verificarli, fornirli al cliente) ma non deve — e non può — modificarli.

Obiettivi della funzionalità

- **Sola lettura:** i verbali restano di proprietà di ICMVerbali; ICMFatturazioni li *consulta* soltanto, senza mai crearli, modificarli o rigenerarne il PDF.
- **Ricerca guidata** per Cliente → Attività → Cantiere, con un filtro a cascata a tre livelli coerente con le altre maschere dell'applicativo.
- **Visibilità governata dallo stato reale:** si mostrano solo i verbali *firmati* dal Coordinatore per la Sicurezza in Esecuzione (CSE) il cui PDF esiste fisicamente sul disco.
- **Messa a disposizione del PDF:** apertura del singolo verbale in una nuova scheda e download in blocco (ZIP) di tutti i verbali del filtro corrente.
- **Sicurezza:** accesso riservato agli utenti autenticati; nessun file raggiungibile in forma anonima.

Principio di fondo

ICMFatturazioni non è la fonte di verità dei verbali. Ne è un *lettore*: interroga solo lo schema `fatt.*` del database condiviso e legge i PDF già prodotti da ICMVerbali dal filesystem comune. Non esiste alcun punto del codice che scriva o generi un verbale.

IL MODELLO DI CONDIVISIONE

4 Contesto: database unificato e entità condivise

Per capire come ICMFatturazioni legge i verbali serve inquadrare il modello di convivenza fra i due applicativi sul database `ICMVerbalIdb`. La regola architetturale è netta: **ICMFatturazioni interroga esclusivamente lo schema `fatt.*`**, mai direttamente le tabelle `dbo.*` di proprietà di ICMVerbali.

Le entità che i due mondi hanno in comune sono esposte a ICMFatturazioni tramite **viste** nello schema `fatt`, che rinominano colonne e nascondono i dettagli del dominio verbali. La tabella seguente riassume la corrispondenza rilevante per questa funzionalità.

Concetto (fatturazioni)	Vista fatt.*	Tabella reale (ICMVerbali)	Note
Anagrafica / Cliente	fatt.Anagrafica	dbo.Committente	Il committente del progetto
Attività	fatt.Attivita	dbo.Progetto	Codice = Numero, Nome = Descrizione
Cantiere	fatt.Cantiere	dbo.Cantiere	Appartiene a un progetto
Verbale (consultazione)	fatt.VerbaliConsultazione	dbo.Verbale (+ join)	Vista di sola lettura, oggetto di questo documento

La catena del dominio verbali è gerarchica: un **Committente** ha uno o più **Progetti**, ciascun progetto uno o più **Cantieri**, e ogni cantiere uno o più **Verbali**. Il verbale punta direttamente solo al cantiere: cliente e attività si ricavano risalendo i join.



Figura 1 — La catena Committente → Progetto → Cantiere → Verbale e la sua mappatura sui concetti di fatturazione.

Vincolo di ownership

Lo schema è di proprietà di ICMVerbali: la vista `fatt.VerbaliConsultazione` è stata creata come *migration numerata* (la 073) nel repository di ICMVerbali, non in ICMFatturazioni, la cui cartella `Migrations/` è congelata.

VISIONE D'INSIEME

5 Architettura della soluzione

La funzionalità rispetta l'architettura a strati (N-Tier) dell'applicativo: **UI** → **Manager** → **Repository** → **Database**. A questa si affianca un secondo canale, parallelo e distinto, per i **file PDF**: i metadati dei verbali arrivano dal database, ma il contenuto binario dei PDF arriva dal **filesystem condiviso** tramite un servizio dedicato (`IVerbaleReportStorage`).

Sono quindi in gioco **due sorgenti**: il DB (via Dapper) per sapere *quali* verbali esistono e con quali attributi, e il disco per leggere *il PDF* vero e proprio. Un verbale è mostrabile solo se entrambe le sorgenti concordano.

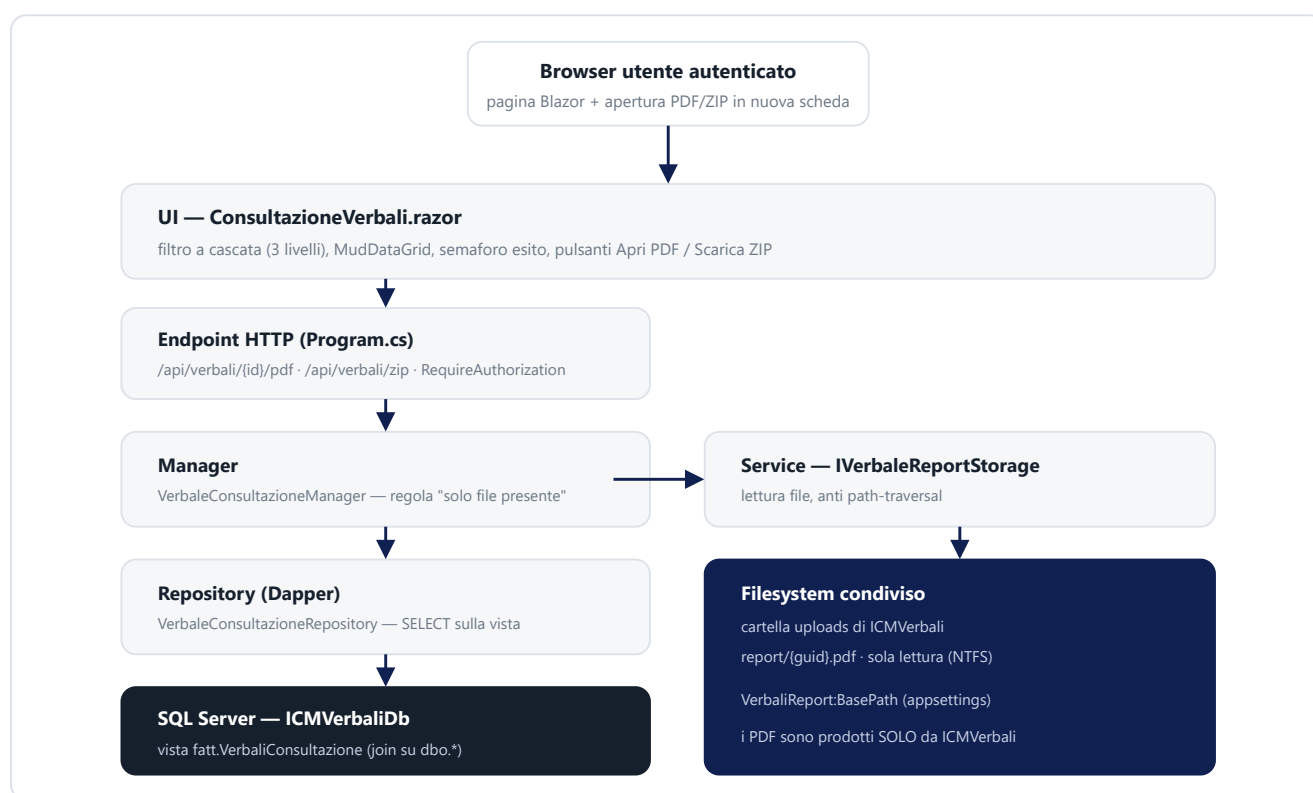


Figura 2 — Architettura a due canali: i metadati scendono per lo stack N-Tier fino al DB; i PDF passano dal Manager al servizio di storage sul filesystem condiviso.

Perché due canali separati

Tenere i PDF fuori dal database (nessun BLOB) mantiene ICMVerbal unica proprietaria dei file e permette a ICMFatturazioni un accesso in sola lettura, revocabile a livello di permessi NTFS, senza toccare né duplicare i binari.

SORGENTE DEI METADATI

6 Il modello dati e la vista `fatt.VerbaliConsultazione`

Il cuore della condivisione è una **vista di sola lettura** creata dalla migration **073** di ICMVerbali. Incapsula tutti i join del dominio verbali e pubblica solo le colonne che servono alla maschera, già filtrate ai soli verbali **firmati e non cancellati**.

Definizione della vista (estratto)

```
CREATE OR ALTER VIEW fatt.VerbaliConsultazione
AS
SELECT
    v.Id                AS IdVerbale,
    v.Numero, v.Anno, v.Data,
    cm.Id              AS IdAnagrafica,
    cm.RagioneSociale AS RagioneSocialeCliente,
    p.Id               AS IdAttivita,
    p.Codice           AS NumeroAttivita,
    p.Nome             AS DescrizioneAttivita,
    c.Id               AS IdCantiere,
    c.Ubicazione       AS UbicazioneCantiere,
    cse.Nominativo     AS CseNominativo,
    imp.RagioneSociale AS ImpresaAppaltatrice,
    ce.Etichetta       AS EsitoEtichetta,
    ce.Severita        AS EsitoSeverita,
    v.ReportPath       AS ReportPath,
    (SELECT COUNT(*) FROM dbo.PrescrizioneCse pr
     WHERE pr.VerbaleId = v.Id) AS NumeroPrescrizioni
FROM dbo.Verbale v
    INNER JOIN dbo.Cantiere      c  ON c.Id = v.CantiereId
    INNER JOIN dbo.Progetto      p  ON p.Id = c.ProgettoId
    INNER JOIN dbo.Committente   cm ON cm.Id = p.CommittenteId
    LEFT JOIN dbo.Persona        cse ON cse.Id = v.CsePersonaId
    LEFT JOIN dbo.ImpresaAppaltatrice imp ON imp.Id = v.ImpresaAppaltatriceId
    LEFT JOIN dbo.CatalogoEsito  ce  ON ce.Id = v.EsitoId
WHERE v.IsDeleted = 0
      AND v.Stato > 0;  -- Stato 0 = bozza; >0 = firmato
```

Osservazioni chiave sulla vista:

- I tre `INNER JOIN` (cantiere, progetto, committente) sono obbligatori: senza quella catena il verbale non è collocabile e non deve comparire.
- I `LEFT JOIN` su CSE, impresa ed esito sono opzionali: un verbale può non avere ancora un esito o un'impresa valorizzati, e la riga rimane comunque valida.
- `Severita` proviene da `dbo.CatalogoEsito` e alimenta il semaforo di conformità nella UI (0 = Conforme ... 3 = Sospensione).
- Il filtro `Stato > 0 AND IsDeleted = 0` esclude a monte bozze e verbali cancellati: la maschera non li vedrà mai.

Il read-model lato applicazione

Alla vista corrisponde in C# il record immutabile `VerbaleConsultazione` (namespace `Models`), una riga della griglia. Include gli Id dei tre livelli di filtro perché la UI costruisce gli universi dei selettori e restringe la griglia interamente in memoria (i volumi sono piccoli).

```
public sealed record VerbaleConsultazione
{
    public required Guid    IdVerbale { get; init; }
    public int?             Numero    { get; init; }
    public int?             Anno      { get; init; }
    public required DateOnly Data     { get; init; }

    // Livello 1 - anagrafica
    public required Guid    IdAnagrafica          { get; init; }
    public required string  RagioneSocialeCliente { get; init; }
    // Livello 2 - attività
    public required Guid    IdAttivita            { get; init; }
    public required string  NumeroAttivita        { get; init; }
    public required string  DescrizioneAttivita    { get; init; }
    // Livello 3 - cantiere
    public required Guid    IdCantiere            { get; init; }
    public required string  UbicazioneCantiere    { get; init; }

    public string?  CseNominativo      { get; init; }
    public string?  ImpresaAppaltatrice { get; init; }
    public string?  EsitoEtichetta      { get; init; }
    public SeveritaEsito? EsitoSeverita { get; init; }
    public int      NumeroPrescrizioni { get; init; }

    // Percorso RELATIVO del PDF sotto la cartella uploads di ICMVerbali
    public string? ReportPath { get; init; }

    public string NomeVerbale =>
        Numero is int n ? $"Verbale {n}/{Anno}" : "Verbale (s.n.)";
}
```

La gravità dell'esito è modellata dall'enum `SeveritaEsito : byte`, mirror 1:1 del catalogo di ICMVerbali, con valore di persistenza esplicito:

Valore	Enum	Semaforo	Significato
0	Conforme	Verde	Nessuna non conformità rilevata
1	NonConformitaMinori	Giallo	Non conformità minori
2	NonConformitaGravi	Arancio	Non conformità gravi
3	Sospensione	Rosso	Sospensione dell'attività di cantiere
—	null	Grigio	Esito non impostato

DATA ACCESS LAYER

7 Data Access Layer (repository Dapper)

Il `VerbaleConsultazioneRepository` è un repository Dapper minimale: espone un solo metodo di lettura, `GetEsportabiliAsync`, che restituisce i *candidati* all'export — cioè i verbali della vista che hanno un `ReportPath` valorizzato.

```
private const string SqlSelectEsportabili = ""
    SELECT IdVerbale, Numero, Anno, Data,
           IdAnagrafica, RagioneSocialeCliente,
           IdAttivita, NumeroAttivita, DescrizioneAttivita,
           IdCantiere, UbicazioneCantiere,
           CseNominativo, ImpresaAppaltatrice,
           EsitoEtichetta, EsitoSeverita,
           NumeroPrescrizioni, ReportPath
    FROM fatt.VerbaliConsultazione
    WHERE ReportPath IS NOT NULL
    ORDER BY Data DESC, Numero DESC;
    "";
```

Dettagli implementativi degni di nota:

- Il repository interroga **solo** lo schema `fatt.*` (la vista), mai `dbo.*`: la separazione di ownership è rispettata anche nel codice.
- La colonna `Data` è un `DATE` SQL: Dapper non la mappa automaticamente su `DateOnly` in lettura, quindi un DTO interno `Row` la legge come `DateTime` e la converte in `DateOnly` nel metodo `ToModel`. Analogamente `EsitoSeverita` è un `tinyint NULL` letto come `byte?` e proiettato sull'enum.
- Il filtro `WHERE ReportPath IS NOT NULL` scarta i firmati "legacy" privi di archivio: sono candidati inutili perché non avrebbero un file da servire.

Perché "candidati" e non "risultato definitivo"

Il repository sa che il `ReportPath` è valorizzato, ma non che il file *esista davvero* su disco. Quella verifica — che richiede l'accesso al filesystem — è responsabilità del Manager (capitolo 8). La divisione delle responsabilità è netta: il repository parla solo col database.

BUSINESS LOGIC

8 Business logic: il Manager e il filtro di esistenza fisica

Il `VerbaleConsultazioneManager` aggiunge ai candidati del repository la regola di dominio più importante della funzionalità: **si mostra un verbale solo se il suo PDF esiste fisicamente sul disco, e non lo si rigenera mai**.

```
public async Task<IReadOnlyList<VerbaleConsultazione>> ElencoEsportabiliAsync(
    CancellationToken cancellationToken = default)
{
    var candidati = await _repository.GetEsportabiliAsync(cancellationToken);
    // Filtro di esistenza fisica: un ReportPath valorizzato ma senza
    // file su disco è dato sporco/legacy → escluso (mai rigenerato).
    return candidati.Where(v => _reportStorage.Esiste(v.ReportPath)).ToList();
}
```

Il metodo `ElencoPerFiltroAsync(idAnagrafica, idAttivita?, idCantiere?)` riusa lo stesso elenco esportabile e lo restringe ai tre livelli del filtro; è quello che alimenta la generazione dello ZIP lato endpoint.

Il triplo filtro di visibilità

La decisione "questo verbale è mostrabile" nasce dalla combinazione di **tre condizioni applicate in tre punti diversi** dell'architettura. È una difesa a strati: ogni livello aggiunge un vincolo che il precedente non poteva verificare.

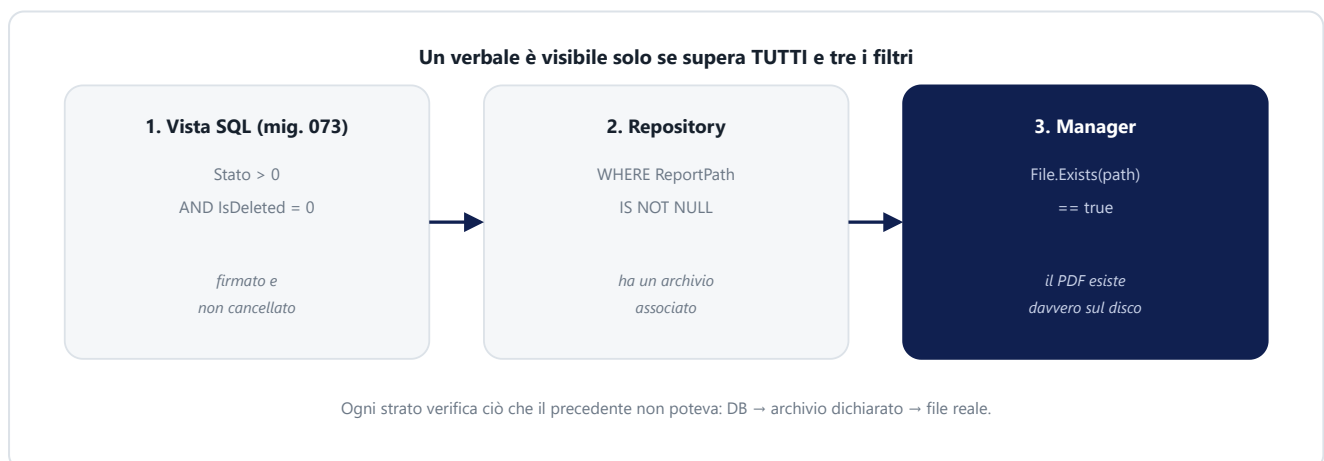


Figura 3 — Il triplo filtro di visibilità distribuito su vista SQL, repository e manager.

Il Manager è registrato come servizio di sola lettura: **nessun audit** (la Regola di audit si applica alle modifiche dati, qui assenti) e nessun tracciamento se non il log degli eventuali errori.

ACCESSO AI FILE

9 Storage dei PDF sul filesystem condiviso

Il servizio `IVerbaleReportStorage` astrae l'accesso ai PDF. Oggi la sorgente è il **filesystem condiviso** — la cartella `uploads` di ICMVerbalì, letta in sola lettura — ma l'interfaccia isola questa scelta: una futura sorgente (per esempio un endpoint HTTP di ICMVerbalì) non impatterebbe Manager, endpoint o UI.

```
public interface IVerbaleReportStorage
{
    // True se il PDF esiste fisicamente.
    bool Esiste(string? reportPath);

    // Byte del PDF, oppure null se il file non c'è (mai rigenerato).
    Task<byte[]?> LeggiAsync(string? reportPath, CancellationToken ct = default);
}
```

Configurazione: `VerbalìReport:BasePath`

La radice dello storage è configurabile in `appsettings` (sezione `VerbalìReport`). Punta alla cartella `uploads` di ICMVerbalì, quella che contiene la sottocartella `report` con i PDF firmati. Un `ReportPath` del DB ha forma `report/{guid}.pdf` e viene risolto sotto questa radice.

- **In sviluppo** il valore reale sta in `appsettings.Development.json` e punta alla cartella locale di ICMVerbalì.
- **In produzione** le due applicazioni convivono sullo stesso server: si dà all'App Pool di ICMFatturazioni un permesso NTFS di **sola lettura** sulla cartella `uploads` di ICMVerbalì.
- **Vuoto** = storage non configurato: nessun verbale sarà scaricabile (comportamento di sicurezza per difetto, non un errore).

Difesa contro il path traversal

Sebbene il `ReportPath` provenga dal database (quindi da fonte affidabile), lo storage applica comunque una verifica a costo nullo: il percorso risolto deve restare **dentro** `BasePath`. Se un dato anomalo tentasse di uscire dalla cartella (es. con `../`), il percorso viene rifiutato e loggato.

```
private string? RisolviPathSicuro(string? reportPath)
{
    if (string.IsNullOrEmpty(_basePathAssoluto) ||
        string.IsNullOrWhiteSpace(reportPath))
        return null;

    var relativo = reportPath
        .Replace('/', Path.DirectorySeparatorChar)
        .TrimStart(Path.DirectorySeparatorChar);
    var combinato = Path.GetFullPath(Path.Combine(_basePathAssoluto, relativo));

    // Il file deve restare sotto BasePath (anti path-traversal).
    var radice = _basePathAssoluto.TrimEnd(Path.DirectorySeparatorChar)
        + Path.DirectorySeparatorChar;
    if (!combinato.StartsWith(radice, StringComparison.OrdinalIgnoreCase))
    {
        _logger.LogWarning("ReportPath fuori da BasePath ignorato: {ReportPath}", reportPath);
        return null;
    }
    return combinato;
}
```

Regola invariante

ICMFatturazioni **non genera mai** il PDF di un verbale. `LeggiAsync` restituisce `null` per file assente — un caso previsto, non un'eccezione — e la maschera si limita a non mostrare quel verbale. La rigenerazione è competenza esclusiva di ICMVerbalì.

MESSA A DISPOSIZIONE DEI FILE

10 Gli endpoint HTTP: apertura PDF e download ZIP

La UI non serve i file direttamente: apre in una nuova scheda due **Minimal API** registrate in `Program.cs`. Entrambe sono protette (capitolo 12).

10.1 — Apertura del singolo PDF

`GET /api/verbali/{id}/pdf` restituisce il PDF del verbale in modalità *inline* (si apre nel visualizzatore del browser). La logica riusa `ElencoEsportabiliAsync`, che **già garantisce** "firmato + file presente": se l'id non è tra gli esportabili, la risposta è `404`.

```
app.MapGet("/api/verbali/{id:guid}/pdf", async (
    Guid id, HttpContext httpContext,
    IVerbaleConsultazioneManager verbaliManager,
    IVerbaleReportStorage reportStorage,
    ILoggerManager logManager, CancellationToken ct) =>
{
    var esportabili = await verbaliManager.ElencoEsportabiliAsync(ct);
    var verbale = esportabili.FirstOrDefault(v => v.IdVerbale == id);
    if (verbale is null) return Results.NotFound();

    var pdf = await reportStorage.LeggiAsync(verbale.ReportPath, ct);
    if (pdf is null) return Results.NotFound(); // race: file rimosso nel frattempo

    httpContext.Response.Headers.ContentDisposition =
        $"inline; filename=\"Verbale_{verbale.Numero}-{verbale.Anno}.pdf\"";
    return Results.File(pdf, "application/pdf");
})
.RequireAuthorization();
```

10.2 — Download in blocco (ZIP)

`GET /api/verbali/zip?idAnagrafica=...&idAttivita=...&idCantiere=...` impacchetta in un unico ZIP **tutti** i verbali del filtro corrente (non solo la pagina visibile a schermo). I parametri viaggiano in query string: `idAnagrafica` è obbligatorio, gli altri due opzionali restringono al livello selezionato.

- L'archivio è costruito in memoria (`ZipArchive` su `MemoryStream`): adeguato ai volumi in gioco (decine di PDF).
- I nomi `Verbale_N-AAAA` possono ripetersi tra cantieri diversi: un helper li disambigua con un suffisso progressivo dentro lo ZIP.
- Un file sparito nel frattempo viene **saltato**, non blocca l'export; se *tutti* i file risultano assenti, la risposta è `404`.

Il flusso completo di una richiesta di apertura PDF è il seguente.

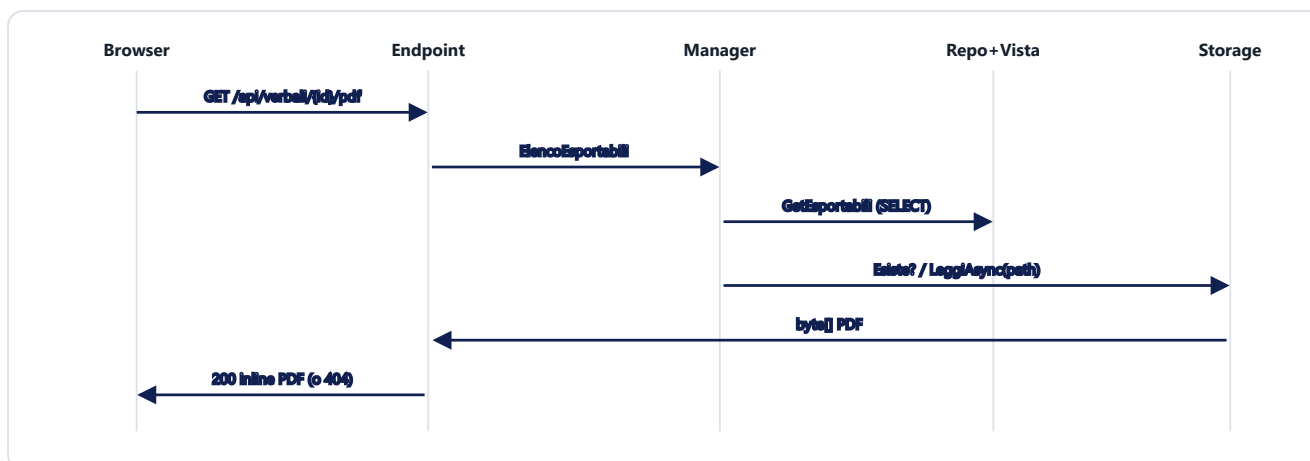


Figura 4 — Sequenza di una richiesta di apertura PDF: l'endpoint valida via Manager (firmato + presente) e serve i byte letti dallo storage.

INTERFACCIA UTENTE

11 L'interfaccia utente: filtro a cascata e griglia

La pagina `ConsultazioneVerbali.razor` replica lo schema delle maschere sorelle (Stampe fatture, Stampa scadenze): tre selettori in cascata in alto e una griglia dei risultati sotto. All'avvio carica **una sola volta** l'universo dei verbali esportabili e da quello deriva quali clienti, attività e cantieri hanno effettivamente verbali; il resto del filtraggio avviene in memoria.

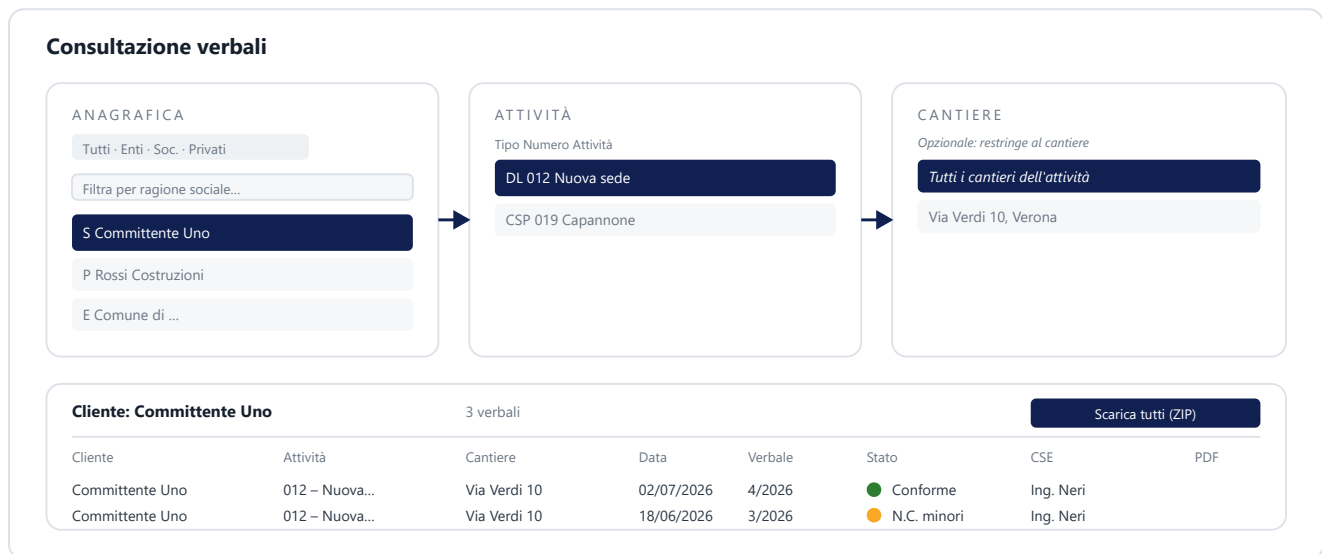


Figura 5 — Schema della maschera: i tre selettori in cascata Anagrafica → Attività → Cantiere e la griglia dei verbali con semaforo esito e azioni PDF/ZIP.

Comportamento del filtro a cascata

- **Anagrafica:** radio Tutti / Enti / Soc. / Privati + ricerca per ragione sociale. Sono elencati solo i clienti con almeno un verbale esportabile.
- **Attività:** si popola solo dopo aver scelto un cliente; mostra le attività di quel cliente che hanno verbali.
- **Cantiere:** opzionale. "Tutti i cantieri dell'attività" mantiene la griglia a livello attività; scegliendo un cantiere la si restringe ulteriormente.
- Cambiando un livello a monte si azzerano i livelli a valle; cambiando il radio del tipo, se il cliente selezionato non vi rientra più, la cascata si resetta.

La griglia dei risultati

È una `MudDataGrid` virtualizzata con la prima colonna (Cliente) sticky come "chiave umana". Colonne: Cliente, Attività, Cantiere, Data, Verbale (N/anno), **Stato** (pallino colorato per gravità dell'esito + etichetta), CSE, Impresa, numero prescrizioni, più l'icona *Apri PDF*. Il colore del semaforo è calcolato dalla funzione pura `ColoreEsito(SeveritaEsito?)`.

Le azioni chiamano gli endpoint via JavaScript interop, aprendo una nuova scheda del browser:

```
// apertura singolo PDF
await JS.InvokeVoidAsync("open", $"/api/verbali/{v.IdVerbale}/pdf", "_blank");

// download ZIP del filtro corrente
var url = $"/api/verbali/zip?idAnagrafica={_anagraficaSelezionata.IdAnagrafica}";
if (_attivitaSelezionata is not null) url += "&idAttivita={_attivitaSelezionata.IdAttivita}";
if (_cantiereSelezionato is not null) url += "&idCantiere={_cantiereSelezionato.IdCantiere}";
await JS.InvokeVoidAsync("open", url, "_blank");
```

Tutti i percorsi di errore della pagina (caricamento iniziale, attività, cantieri, apertura PDF, download ZIP) sono avvolti in `try/catch` che loggano via `ILogManager` e mostrano uno snackbar all'utente: nessun errore resta silenzioso.

PROTEZIONE DEGLI ACCESSI

12 Sicurezza, permessi e messa in produzione

Una domanda legittima è: **gli endpoint che servono i verbali sono protetti, o chiunque può accedervi?** La risposta è che l'accesso è riservato agli utenti autenticati, con una doppia difesa.

12.1 — Autenticazione obbligatoria (doppia difesa)

- **Policy globale di fallback:** l'intera applicazione richiede `RequireAuthenticatedUser()`. Solo le pagine marcate esplicitamente `[AllowAnonymous]` (login, reset password) sono aperte. Un utente non autenticato riceve un redirect/401 e non scarica nulla.
- **Vincolo esplicito sull'endpoint:** entrambe le Minimal API dei verbali terminano con `.RequireAuthorization()`. Anche se la policy globale fosse allentata, gli endpoint resterebbero comunque protetti.

```
builder.Services
    .AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build());
// ...
app.MapGet("/api/verbali/{id:guid}/pdf", /* ... */.RequireAuthorization());
app.MapGet("/api/verbali/zip", /* ... */.RequireAuthorization());
```

Livello di protezione attuale — nota di trasparenza

Oggi la protezione è *autenticazione*, non *autorizzazione per ruolo*: qualsiasi utente loggato che veda la voce di menu può aprire qualsiasi verbale conoscendone l'id (gli id sono GUID, quindi non enumerabili). Non è imposta segregazione per cliente. Se in futuro servisse limitare l'accesso a un permesso specifico o per cliente, il punto di intervento è aggiungere una policy dedicata su questi due endpoint — l'infrastruttura di ruoli/permessi dinamici è già presente.

12.2 — Il perimetro esterno: i permessi NTFS

Il secondo perimetro di sicurezza è il filesystem. In produzione, con le due applicazioni sullo stesso server, l'App Pool di ICMFatturazioni deve avere permesso di **sola lettura** sulla cartella `uploads` di ICMVerbali. Nessun permesso di scrittura è necessario né va concesso: ICMFatturazioni legge e basta.

```
icacls "C:\...\ICMVerbali\...\App_Data\uploads" ^
/grant "IIS AppPool\ICMFatturazioni:(OI)(CI)(RX)"
```

La combinazione dei due perimetri — autenticazione applicativa + sola lettura NTFS — fa sì che i PDF non siano raggiungibili in forma anonima e non siano in alcun modo modificabili da ICMFatturazioni.

12.3 — Gestione degli errori negli endpoint

Gli endpoint gestiscono i fallimenti in modo esplicito: cartella non accessibile all'App Pool o file rimosso danno luogo a un log arricchito con la spiegazione tecnica probabile (`ILogManager.LogErroreAsync`) e a una risposta `500` con messaggio neutro per l'utente. Il caso "verbale non trovato / non esportabile" è invece un `404`, distinto da un errore.

13 Dependency Injection, configurazione e test

Registrazione dei servizi

I componenti sono registrati in `Program.cs` secondo le convenzioni del progetto: repository e manager come *scoped*, lo storage come *singleton* (è stateless a runtime), con il binding delle opzioni sulla sezione di configurazione.

```
builder.Services.AddScoped<IVerbaleConsultazioneRepository, VerbaleConsultazioneRepository>();
builder.Services.AddScoped<IVerbaleConsultazioneManager, VerbaleConsultazioneManager>();

builder.Services.Configure<VerbaliReportOptions>(
    builder.Configuration.GetSection(VerbaliReportOptions.SectionName));
builder.Services.AddSingleton<IVerbaleReportStorage, VerbaleReportStorage>();
```

Configurazione (appsettings)

```
"VerbaliReport": {
  "BasePath": "" // vuoto in prod versionato; valore reale in Development / env
}
```

La voce di menu

La voce "Consultazione verbali" è inserita dalla stessa migration 073, nel gruppo "Attività studio" (ordine 21, subito dopo "Documenti XML"), con icona `FactCheck`. Admin e Superadmin la vedono senza necessità di grant espliciti, come le altre voci del gruppo.

Copertura di test

La logica di dominio è coperta da unit test con fake manuali (niente librerie di mock, per convenzione di progetto):

- `FakeVerbaleConsultazioneRepository` e `FakeVerbaleReportStorage` simulano rispettivamente i candidati del DB e l'esistenza dei file.
- `VerbaleConsultazioneManagerTests` verifica il comportamento cardine: un verbale con `ReportPath` ma senza file è escluso dall'elenco esportabile; il filtro per i tre livelli restituisce il sottoinsieme corretto.
- La suite complessiva dell'applicativo resta verde (532 test al rilascio di questa funzionalità).

SINTESI

14 Riepilogo e possibili estensioni

La funzionalità mette a disposizione di ICMFatturazioni i verbali di cantiere del dominio ICMVerbali in modo **sicuro, di sola lettura e senza duplicazione**, appoggiandosi al database condiviso per i metadati e al filesystem comune per i PDF.

Aspetto	Scelta implementativa
Sorgente metadati	Vista <code>fatt.VerbaliConsultazione</code> (migration 073, ICMVerbali)
Sorgente PDF	Filesystem condiviso, sola lettura, via <code>IVerbaleReportStorage</code>
Regola di visibilità	Firmato (Stato>0, non cancellato) + ReportPath valorizzato + file esistente
Rigenerazione PDF	Mai: competenza esclusiva di ICMVerbali
Messa a disposizione	Endpoint <code>/api/verbali/{id}/pdf</code> (inline) e <code>/api/verbali/zip</code> (blocco)
Protezione	Autenticazione (policy globale + <code>RequireAuthorization</code>) + sola lettura NTFS
Audit	Assente (sola lettura); solo logging degli errori

Possibili estensioni future

- **Autorizzazione granulare:** legare gli endpoint a un permesso dedicato o segregare i verbali per cliente, sfruttando il sistema di ruoli/menu dinamici già presente.
- **Sorgente PDF alternativa:** grazie all'astrazione `IVerbaleReportStorage`, si potrebbe passare dal filesystem a un endpoint HTTP esposto da ICMVerbali senza toccare Manager, endpoint o UI.
- **Filtri aggiuntivi** in griglia (per intervallo di date, per esito, per CSE), oggi non presenti ma abilitati dal read-model che già trasporta questi campi.

In una frase

ICMFatturazioni consulta i verbali interrogando solo lo schema `fatt.*` e leggendo i PDF dal disco in sola lettura: mostra unicamente ciò che è firmato e realmente presente, lo serve ai soli utenti autenticati, e non scrive né rigenera mai nulla.